Sign Friendly Gloves - Prototype Plan

ID1050A Engineering Design, IIT Palakkad | Group 39 | 3-month, Rs 8,000 prototype plan for a 6-member team

Terminology note

This plan uses **text-to-speech (TTS)**, not speech-to-text. Speech-to-text converts spoken audio into text, which is the opposite direction and not part of this pipeline. Worth fixing in the proposal document as well, since a reviewer is likely to flag it.

The scoping call

Two gloves, a full sign vocabulary, real-time ML, a wearable form factor, Rs 8,000, 3 months, and a 6-person team mostly new to embedded electronics and ML - that is a lot to take on at once. The deliberate cut: **do not aim for full ISL.** Pick 10-15 signs (a mix of useful words and a few alphabet letters), get single-hand recognition solid first, and only extend to two-hand signs if the team is ahead of schedule. A working 12-sign demo beats a half-working 200-sign one, and this kind of scope creep is what typically stalls course projects in month three.

System pipeline

One shared controller serves both gloves instead of duplicating electronics per hand: a 16-channel analog multiplexer lets a single Arduino read all 10 flex sensors, and both IMUs share the I2C bus at different addresses (0x68 / 0x69). This is the single biggest cost and complexity saver in the design.

1. Glove sensors	10 flex sensors + 2 IMUs (motion), one IMU per wrist
2. Wrist hub	Arduino Nano + multiplexer reads all sensors
3. Bluetooth link	HC-05 sends readings to a phone or laptop
4. ML classifier	scikit-learn model (on phone/laptop) predicts the sign
5. Bluetooth link	Sends the recognized word back to the glove
6. Speaker output	DFPlayer Mini speaks the recognized word aloud

Component list (bill of materials)

Prices are current India online-store estimates (Robu.in / Robokits / Amazon.in) as of August 2026 - get quotes before ordering, they can vary about 20% by vendor.

#	Item	Qty	Unit Rs	Subtotal Rs	Why
1	Arduino Nano (CH340 clone)	1	200	200	Central hub - reads everything
2	CD74HC4067 16-ch analog multiplexer	1	90	90	1 Nano reads 10 flex sensors
3	MPU6050 accelerometer/gyro module	2	175	350	One per wrist, I2C
4	HC-05 Bluetooth module	1	250	250	Wireless link to phone/laptop
5	DFPlayer Mini MP3 module	1	200	200	Plays pre-recorded word audio

#	Item	Qty	Unit Rs	Subtotal Rs	Why
6	8-ohm 2-3W mini speaker	1	80	80	Audio output
7	MicroSD card, 8 GB	1	150	150	Stores word clips
8	Velostat/anti-static sheet + copper tape	-	400	400	DIY flex sensors, ~15-18 w/ spares
9	10 k-ohm resistors (pack)	15	~3	50	Voltage dividers
10	Stretch-fit fabric gloves	3 pairs	~85	250	Iteration + final
11	Perfboard, hookup wire, solder, heat-shrink	-	500	500	Assembly
12	Pin headers / JST connectors	-	150	150	Detachable glove-to-hub wiring
13	3.7V Li-ion cell + TP4056 module + switch	1 set	300	300	Portable power for final demo
14	Small enclosure, Velcro, elastic	-	250	250	Wearability
15	Misc (USB cable, jumper wires)	-	300	300	Buffer
	Core total			**Rs 3,520**	

That leaves roughly **Rs 4,500 of the Rs 8,000 budget genuinely spare**. Don't force spending it - put it toward:

- 2-3 real commercial flex sensors (Rs 250-400 each) to cross-check the DIY ones and as backups when one gets damaged mid-solder
- A multimeter (~Rs 400-600) if the team doesn't already own one
- A soldering iron kit (~Rs 300-500) if needed

First-time soldering by six people means burnt components - budget for that reality rather than a perfectly optimized BOM.

Why DIY flex sensors instead of buying them: commercial bend sensors run Rs 250-400 each - times 10 that is most of the budget gone. A velostat-and-copper-tape sandwich costs pennies and is a well-documented technique (some builders even use free anti-static bags instead of buying velostat). The tradeoff is noise - DIY sensors drift and spike more than commercial ones - so per-user calibration and a moving-average filter in software are needed. That is a fair trade for the money saved.

Software: where the ML actually happens

Given the team is still building up Python (functions, data structures, moving into OOP), scikit-learn is the right level - no need for TensorFlow or deep learning here, and it works well on small structured sensor data like this.

```
# training_pipeline.py -- simplified sketch
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
import joblib

data = pd.read_csv("sign_samples.csv")   # columns: flex1..flex10, ax,ay,az (x2), label
X = data.drop(columns=["label"])
y = data["label"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = RandomForestClassifier(n_estimators=100)
```

```
model.fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))
joblib.dump(model, "sign_classifier.pkl")
```

Two things that matter more than the algorithm choice:

- **Include a "rest / no sign" class** in training data. Without it, an idle hand gets randomly classified as some word and the glove blurts things out constantly - the single most common way these projects fail a live demo.
- **Debounce**: only trigger speech after the same prediction shows up for several consecutive readings (e.g. 5 in a row), not on one noisy frame.

Team split (6 people)

- **2 - Hardware:** sensor fabrication, glove assembly, soldering, wiring
- **2 - Firmware:** Arduino code (multiplexer control, sensor reading, calibration, Bluetooth/serial)
- **2 - Software/ML:** Python data collection tool, training pipeline, real-time inference + DFPlayer trigger logic
- **Everyone** contributes training data - with 6 different hand sizes recording samples, the classifier ends up more robust than if one person generated all the data. Worth mentioning in the report as a deliberate design choice, not an afterthought.

Timeline (12 weeks)

Weeks	Milestone
1-2	Finalize 10-15 sign vocabulary, order parts, literature review
3-4	Build & bench-test DIY flex sensors, breadboard the mux + Nano + 2x MPU6050, raw sensor values on serial monitor
5	Mount sensors on gloves, wire up, write calibration routine (min/max flex per finger)
6-7	Data collection - all 6 members record 40-60 samples/sign, including the "rest" class
8	Train classifier, check confusion matrix, fix commonly-confused signs
9	Bluetooth integration + record word audio clips + wire up DFPlayer trigger
10	Full pipeline integration test (glove -> BT -> classify -> speech), debounce tuning
11	Robustness testing across users, buffer for slippage, Gantt/report writeup
12	Final demo rehearsal, submission

This is a tight but doable path - the main risk is not budget, it is time. Protect weeks 6-8 (data collection and training) especially; that is where most projects like this either click or stall.